

Lecture 16: Binary Arithmetic (Addition, Subtraction, Multiplication & Division)

Learning Objectives

By the end of this lecture, students will be able to:

- Understand why computers perform arithmetic in Binary.
 - Perform Binary Addition.
 - Perform Binary Subtraction using the borrow method.
 - Perform Binary Multiplication.
 - Perform Binary Division.
 - Understand Carry and Borrow.
 - Identify Overflow in binary addition.
 - Solve exam-style binary arithmetic questions.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn Binary Arithmetic, including Binary Addition, Binary Subtraction, Binary Multiplication, Binary Division, Carry, Borrow, and Overflow with easy step-by-step examples."

In our previous lectures, we learned about number systems and how to convert between Decimal, Binary, Octal, and Hexadecimal.

But have you ever wondered how a computer actually performs calculations?

When you use a calculator, play a game, or calculate marks in a spreadsheet, the CPU is not adding decimal numbers. It first converts everything into Binary and then performs arithmetic using the Arithmetic Logic Unit (ALU).

In today's lecture, we'll learn Binary Addition, Binary Subtraction, Binary Multiplication, and Binary Division using simple rules and solved examples.

Let's begin.

2. Why Does the Computer Use Binary Arithmetic?

A computer is built from **digital circuits**.

Each circuit has only **two stable states**:

OFF = 0

ON = 1

Because of this:

- Numbers are stored in Binary.
 - Arithmetic is performed in Binary.
 - The ALU is designed to process Binary values.
-

3. Binary Addition

Binary Addition follows rules similar to Decimal Addition, but it uses only **0** and **1**.

Rules of Binary Addition

A	B	Result
0	0	0
0	1	1
1	0	1
1	1	10 (0 with Carry 1)

When adding three bits (including a carry):

A	B	Carry In	Result
1	1	1	11 (1 with Carry 1)

4. Example 1 – Simple Addition

Add:

1010₂

+

0101_2

Solution:

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array}$$

Verification:

$$10 + 5 = 15$$

Binary:

$$1111_2 = 15_{10}$$

5. Example 2 – Addition with Carry

Add:

1111_2

+

0001_2

Solution:

Carry: 1111

$$\begin{array}{r} 1111 \\ + 0001 \\ \hline 10000 \end{array}$$

Explanation:

- $1 + 1 = 10 \rightarrow$ Write 0, Carry 1
- Repeat until the final carry becomes the new leftmost bit.

Answer:

10000_2

6. Carry Concept

A **Carry** occurs when the sum exceeds the value that can be stored in one bit.

Example:

$$1 + 1 = 10$$

- Write: **0**
- Carry: **1**

The carry is added to the next higher bit.

7. Binary Subtraction

Binary Subtraction follows these basic rules:

A	B	Result
0	0	0
1	0	1
1	1	0
0	1	1 (<i>Borrow Required</i>)

When subtracting 1 from 0, you must **borrow** from the next higher bit.

8. Example 1 – Simple Subtraction

Subtract:

1010_2

-

0011_2

Solution:

$$\begin{array}{r} 1010 \\ - 0011 \\ \hline 0111 \end{array}$$

Verification:

$$10 - 3 = 7$$

Answer:

0111_2

9. Example 2 – Borrow Required

Subtract:

1000_2

-

0001_2

Solution:

$$\begin{array}{r} 1000 \\ - 0001 \\ \hline 0111 \end{array}$$

Explanation:

The least significant bit is:

$$0 - 1$$

Since this is not possible, borrow **1** from the next higher bit.

Borrowing changes the value of the current position, allowing the subtraction to continue.

10. Borrow Concept

Borrowing works similarly to Decimal subtraction.

Example:

Decimal:

30

-

You borrow from the tens place.

Binary uses the same idea, but the borrow comes from the next higher binary position.

11. Binary Multiplication

Binary Multiplication is simpler than Decimal Multiplication because multiplication by **1** keeps the value unchanged, and multiplication by **0** results in zero.

Rules

A	B	Result
0×0	0	
0×1	0	
1×0	0	
1×1	1	

12. Example – Binary Multiplication

Multiply:

101_2

×

11_2

Solution:

 101
× 11

 101
+ 1010

 1111

Verification:

$5 \times 3 = 15$

Binary:

1111_2

13. Binary Division

Binary Division follows the same long-division process used in Decimal.

Example

Divide:

1100_2

÷

10_2

Since:

$1100_2 = 12_{10}$

$10_2 = 2_{10}$

Result:

110_2

Verification:

6_{10}

14. Overflow

Overflow occurs when the result requires **more bits** than are available.

Example:

Suppose we have only **4 bits**.

1111_2

+

0001_2

Actual Result:

10000_2

But 5 bits are required.

If the system stores only 4 bits:

0000

The extra carry is lost, causing an **overflow**.

15. Real-Life Analogy

Imagine a row of buckets.

Each bucket can hold only one ball.

- If a bucket already has one ball and another is added, the extra ball moves to the next bucket (**Carry**).
- During subtraction, if a bucket is empty, you borrow a ball from the next bucket (**Borrow**).

This is exactly how binary arithmetic works.

Lecture Summary

Today, we learned how computers perform arithmetic using Binary numbers.

We studied Binary Addition, Binary Subtraction, Binary Multiplication, and Binary Division, along with the concepts of Carry, Borrow, and Overflow. Through solved examples, we saw how the ALU processes Binary values internally to perform calculations.

These concepts are fundamental because every arithmetic operation executed by a processor is ultimately carried out in Binary.

In the next lecture, we'll explore **Signed Number Representation**, including **Unsigned Numbers**, **Sign-Magnitude Representation**, **1's Complement**, and **2's Complement**. These techniques allow computers to represent both positive and negative numbers efficiently.

Thank you for watching, and I'll see you in the next lecture.
